

Laboratório 2 Comparações, Opcodes, Pseudoinstruções**Comparações**

Além do comando `bne`, podemos usar o seu inverso, `beq` (*branch if equal*). Outros saltos condicionais no assembly RISC-V puro fazem comparação de maior ou igual (exemplo: `bge s1, s2, destino`) e menor (`blt s3, s5, destino` vai saltar para destino se $s3 < s5$); ambos fazem a comparação de números sinalizados em complemento de 2 (é o *signed int* da linguagem C). Há versões para *unsigned int* (`bgeu` e `bltu`), que não utilizaremos.

Usando *apenas instruções nativas* do assembly RISC-V, faça um programa que identifica uma de seis diferentes possibilidades de acordo com os valores atuais de s1 e s2. O caso identificado é indicado ao final por s3, de acordo com o seguinte:

s1	s2	s3	s1	s2	s3
s1>=0	s2>32	1	s1<0	s2>32	4
	s2==32	2		s2==32	5
	s2<32	3		s2<32	6

Se precisar fazer um salto incondicional, pode usar a pseudoinstrução b destino ou a pseudoinstrução j destino. (Pseudoinstruções serão vistas à frente).

Código de Máquina

O montador traduz a linguagem *assembly* para o assim chamado código de máquina, que é uma sequência de comandos em binário que o processador compreende. Cada instrução é associada a um número, e cada processador possui seu conjunto de instruções possíveis e regras de formação.

Exemplo: a instrução `add s1, s2, s3` corresponde ao número hexadecimal de 32 bits `0x013904B3` (a notação `0x` para hexadecimal vem da linguagem C).

O core do RISC-V comum (a ISA básica RV32I¹) possui seis formatos que ditam a formação das instruções em código de máquina: R (*register*), I (*immediate*), B (*branch*), U (*upper immediate*), S (*store*), e J (*jump*), dos quais iremos ver apenas os 3 primeiros. As instruções possuem sempre 32 bits e os campos têm posição fixa nos formatos.

Regras de Codificação

O formato R dita a codificação das instruções que fazem operações exclusivamente entre registradores. Abaixo, os números entre parênteses indicam a quantidade de bits dos campos.

	31	25	24	20	19	15	14	12	11	7	6	0
Formato R	funct7 (7)		rs2 (5)		rs1 (5)		funct3 (3)		rd (5)		opcode (7)	

Os sete bits LSB iniciais $b6 \sim b0^2$ formam o campo *opcode* (*operation code*); é o opcode que vai indicar ao processador o formato da instrução. Os campos `rd`, `rs1` e `rs2` identificam os registradores envolvidos e os campos `funct7` e `funct3` são 10 bits adicionais para identificar qual é a instrução.

- 1 Note que o RARS implementa o superset RV32IMFN do RV32I. Este superset inclui (M)ultiplicação e divisão, aritmética de (F)loating point e (N) para “user level interrupts.”
- 2 LSB é *least significant bit*, ou seja, o bit mais à direita, o menos significativo. A convenção que vamos usar é que este é o bit `b0` (pois tem peso $2^0=1$). O MSB (*most significant bit*) é o mais significativo à esquerda, o bit `b31` (pois tem peso $2^{31} = 2\,147\,483\,648 = 2$ Giga).

Exemplo: a instrução `sub s6, t5, s9` faz a subtração $s6 \leftarrow t5 - s9$, e é codificada como 0x419F0B33. Veja o binário:

funct7	rs2	rs1	funct3	rd	opcode
010 0000	1 1001	1 1110	000	1 0110	011 0011

Para entender o mecanismo, consultamos as tabelas ao final deste lab para a instrução `sub`. O opcode de `sub` é 011 0011; os registradores são o destino $rd = s6$ (mapeado para $x22 \Rightarrow 22_{10} = 1\ 0110_2$), o primeiro fonte (*source*) é $rs1 = t5$ (sendo $t5 = x30 \Rightarrow 30_{10} = 1\ 1110_2$) e o segundo registrador fonte é $rs2 = s9$ (com $s9 = x25 \Rightarrow 25_{10} = 1\ 1001_2$); o complemento funct3 é 000 e o funct7 é 010 0000. O resultado final é $0100\ 0001\ 1001\ 1111\ 0000\ 1011\ 0011\ 0011_2 = 419F0B33_{16}$.

Quando o ser humano escreve `sub s6, t5, s9` no RARS, o montador vai interpretar isso e gerar o número 0x419F0B33. Este número é o código de máquina, que será gravado em memória e será entendido pelo processador como a instrução `sub s6, t5, s9` e então executado.

O Formato I com Constantes

O formato I é para instruções que incluem uma constante de 12 bits, dita *imediata* (pois está embutida na instrução), compreendendo *addi*, *branches* e instruções de acesso à memória.

	31	20	19	15	14	12	11	7	6	0
Formato I	imm (12)				rs1 (5)	funct3 (3)	rd (5)	opcode (7)		

Perceba que os campos `rs1`, `funct3`, `rd` e `opcode` estão na mesma posição do formato R, simplificando os circuitos de decodificação.

Como exemplo, a instrução `addi s1, zero, 0x45` vai gerar o código de máquina 0x04500493, sendo $rd = s1 = x9$ e $rs1 = \text{zero}$, com `opcode`=0010011 e `funct3`=000 e `imm`=0000 0100 0101₂=45₁₆. Confira:

imm	rs1	funct3	rd	opcode
0000 0100 0101	0 0000	000	0 1001	001 0011

O código de máquina desta instrução é o número de 32 bits 0000 0010 1101 0000 0000 0100 1001 0011₂ = 04500493₁₆.

Faça manualmente a codificação das seguintes instruções para o código de máquina hexadecimal respectivo, consultando a tabela ao final do arquivo: • <code>add t3, zero, s7</code> • <code>addi s1, s1, 1</code> • <code>sub t6, t5, t4</code> • <code>addi t1, t4, 123</code>	
Escreva as instruções completas executadas pelo processador RISC-V ao encontrar os seguintes códigos de máquina, novamente consultando a tabela manualmente. • 0x0169 02B3 • 0x0576 0693 • 0x0109 E413	

Note que o formato I tem a limitação de que a constante tem no máximo 12 bits; com utilizamos números sinalizados (*signed*) em complemento de 2 por default, isso nos dá uma constante com limites entre 2^{-11} e $2^{11}-1$, ou seja, $-2048 \leq \text{imm} \leq 2047$. Veremos mais sobre isso depois.

Acesso à Memória

A instrução nativa `lw reg1, cte(reg2)` (*load word*) irá ler 32 bits do endereço de memória dado por `reg2+cte` e vai armazenar esse valor em `reg1`. A instrução `sw reg1, cte(reg2)` (*store word*) irá escrever o valor de `reg1` no endereço de memória dado por `reg2+cte`.

Exemplos: `lw s4, 0x400(zero)` carrega para `s4` o valor presente no endereço `0x0000 0400`; `sw t2, 8(s2)` armazena o valor de `t2` no endereço `8+s2`.



Importante: cada endereço de memória armazena 1 byte (são 8 bits), o que é uma convenção quase universal. Portanto, cada dado de 32 bits ocupa 4 endereços de memória.

De forma razoável, os endereços acessados por `lw` e `sw` devem ser múltiplos de 4 (no jargão, devem estar alinhados em 4 bytes), senão ocorrerá um erro de execução.

Atenção! No simulador, deve-se utilizar a área de memória convencionada para RAM. No RARS, podemos usar *acima do endereço 0x1000 0000* na configuração padrão (e até `0x8000 0000`). Note também que a constante `cte` tem apenas 12 bits (!) por usar o formato I, ou seja: *não existe* uma instrução nativa tal como `sw t6, 0x12345678(zero)` porque a constante `0x1234 5678` não cabe em 12 bits (exigiria 32 bits). Isso é uma pseudoinstrução *assemblada*³ para duas instruções nativas (cf. próx. seção).



Teste um programa com a seguinte instrução; monte e tente rodar, observando o erro na aba *Execute* do RARS, no terceiro painel inferior (aba *Messages/Run I/O*). Este erro é devido à falta de alinhamento em endereços múltiplos de 4.

```
li    s3, 0x12345678 # são pseudoinstruções
li    t1, 0x10010034 # (exceto a última)
sw    s3, (t1)        # <= observe o efeito na memória!
lw    t2, 1(t1)       # aqui vai falhar por endereço inválido
```

Constantes de 32 bits e Pseudoinstruções

Se desejamos carregar algum registrador com uma constante de 32 bits, e.g. `s2 ← 0x12345678`, precisaremos de mais de uma instrução (claro, porque cada instrução tem apenas 32 bits para codificar tudo...). Eis como é feito na linguagem assembly RISC-V⁴:

```
lui    s2, 0x12345        # load upper immediate
addi   s2, s2, 0x678
```

A instrução `lui` carrega os 20 bits superiores da constante em `s2`, apagando os 12 bits LSB, de forma que a soma `addi` vai determinar os 12 bits inferiores. Esta instrução usa o formato U, descrito em uma seção opcional mais à frente.

Como isto é muito útil para programação, o montador fornece uma *pseudoinstrução* `li` que funciona como um *alias* (um apelido) para estas duas instruções; o código acima pode ser escrito no RARS como:

```
li    s2, 0x12345678 # carrega s2 com a cte 0x12345678
```

E o montador vai fazer a tradução. Note se se a constante for “pequena” (couber em 12 bits), ele traduz diferente:

³ Nunca use o verbo “assemblar,” isso é um anglicismo. Prefira “montar” (de montador), que é o correto em português.

⁴ Há um caso especial quando o bit b11 da constante do `addi` está setado: a extensão de sinal vai subtrair 1 dos bits superiores quando a soma ocorrer. O montador compensa isso, carregando 1 a mais no `lui`. Não se preocupe com isso.

`li t3,0x7B5` será convertido para `addi t3,zero,0x7B5`

Coisa similar é feita para saltos condicionais: instruções não nativas podem ser facilmente “simuladas” através de pseudoinstruções. Assim, uma pseudoinstrução como `bgt t4,t5,dest` (*branch if greater than* ou “salte para dest se $t4 > t5$ ”) não existe nativamente mas pode ser facilmente realizada sendo traduzida para `blt t5,t4,dest`.

Outras pseudoinstruções interessantes são `mv` (copia um registrador para o outro), `j` (salto incondicional para algum label) e `nop` (*no operation*, instrução que não faz nada).



Reescreva o programa inicial (das comparações de `s1` e `s2`) utilizando *pelo menos uma* instrução `blez` e *pelo menos uma* instrução `beqz`. Observe na aba *Execute* do RARS quais são as instruções nativas equivalentes que o montador gerou.

[Opcional] Bytes na Memória

A ordem dos bytes de uma word de 32 bits na memória é chamada *endianness*, e pode ser “big” ou “little” ou flexível, dependendo da implementação. Veja o *snippet* abaixo:

```
li t5,0x12345678
li s2,0x10010034
sw t5,0(s2)
```

Não é muito importante vocês saberem, mas a arquitetura RISC-V é *little endian*, portanto a memória vai ficar assim depois do programa acima:

endereço	conteúdo
0x10010033	...
0x10010034	0x78
0x10010035	0x56
0x10010036	0x34
0x10010037	0x12
0x10010038	...

A ideia é “little end first,” ou seja, o byte menos significativo vem antes na memória. Poucos sistemas contemporâneos utilizam *big endianness*.



A instrução `lb t1,0(t2)` vai ler *apenas um byte* do endereço dado por `t2`, que não precisa estar alinhado. O byte vai ter seu sinal estendido e armazenado em 32 bits em `t1`.
A instrução `sb` vai escrever os 8 bits LSB do operando no endereço especificado.
Faça então um programa usando `lb` que inverte a *endianness* de um número armazenado no endereço 0x1001 0034 (ou seja, no exemplo anterior, ficaríamos com 0x12 no 1º, 0x34 no 2º...

[Opcional] Formato U de Instruções

Ainda a ser escrito. Consulte o *Guia Prático RISC-V*,⁵ a figura 2.3 tem um resumo da coisa.

Exercícios (Entregar dois no Moodle)

Construa e teste programas que acessam a memória, colocando números para teste através da janela *Data Segment* (a do meio) na tela de *debug* do RARS. Lembre de usar a memória acima do endereço 0x10010000, que é o padrão da janela.

⁵ Tem nas referências no Moodle, mas fica em <http://riscvbook.com/portuguese/> se você quiser.

1. Quais são os maiores valores imediatos que podem ser usados em uma instrução addi? (Por ex., `addi s2, zero, 0x123456` não funciona; o RARS diz “*operand is out of range*” porque a constante tem bits em excesso).
2. Armazene uma sequência crescente de 1000 valores na memória a partir do endereço 0x1001 1234.
3. Transfira 40 words de 32 bits a partir do endereço 0x1001 0100 para a região de memória iniciada em 0x1001 1100. Insira alguns dados para poder visualizar a simulação ocorrendo.
4. Ordene 20 números inteiros.
5. Idem ao 2, mas invertendo a ordem dos dados no destino (a 1ª word se torna a última etc.).
6. Calcule a soma de todos os dados entre 0x1000 0000 e 0x1000 FFFC (inclusive).

Tabela Parcial para Construção de Código de Máquina RISC-V (RV32I)

Convenções de Regs.

x0

zero

x1

ra

x2

sp

x3

gp

x4

tp

x5 - x7

t0 - t2

x8 - x9

s0 - s1

x10 - x17

a0 - a7

x18 - x27

s2 - s11

x28 - x31

t3 - t6

Formatos de Instruções

I

31

20

19

15

14

12

11

7

6

0

imm

(12)

rs1

(5)

funct3

(3)

rd

(5)

opcode

(7)

R

31

25

24

20

19

15

14

12

11

7

6

0

funct7

(7)

rs2

(5)

rs1

(5)

funct3

(3)

rd

(5)

opcode

(7)

B

31

25

24

20

19

15

14

12

11

7

6

0

imm1

(7)

rs2

(5)

rs1

(5)

funct3

(3)

imm2

(5)

opcode

(7)

U, S, J: não usaremos

Tabela de codificação

Mnemônico	Opcode	funct3	funct7	Tipo
add	0110011	000	0000000	R
addi	0010011	000		I
beq	1100011	000		B
bge	1100011	101		B
blt	1100011	100		B
bne	1100011	001		B
ecall	1110011	000		I
lui	0110111			U
lw	0000011	000		I
ori	0010011	110		I
sub	0110011	000	0100000	R
sw	0100011	010		S